

Projet CHILD : Documentation pour le calcul des enfants issus de la cohorte ELFE

Auteur : Gwendall Petit ([UMRAE - gwendall.petit@ec-nantes.fr](mailto:gwendall.petit@ec-nantes.fr))

Mise à jour : 03/2025

Sommaire

Projet CHILD : Documentation pour le calcul des enfants issus de la cohorte ELFE

Introduction

Objectifs

Données d'entrée

Outils

1. Préparation des données

Définition des zones de travail

Traitements dans une base de données

Processus

Séparation des fichiers par enfants

Conformité des fichiers .geojson

Renommage des fichiers

2. Organisation des données et fichiers

Dossier `code`

Dossier `geoclimate`

Dossier `input_data`

Ajout du dossier vide `creation`

Dossier `noisemodelling`

Dossier `output_data`

Modèle de données

3. Exécution des calculs

4. Mise en forme des résultats

Compression des bâtiments

Unification des enfants

Introduction

Dans le cadre du projet de recherche CHILD, cette documentation décrit l'enchaînement des traitements et opérations qui permettent de calculer les niveaux sonores auxquels sont exposés les enfants issus de la [cohorte ELFE](#).

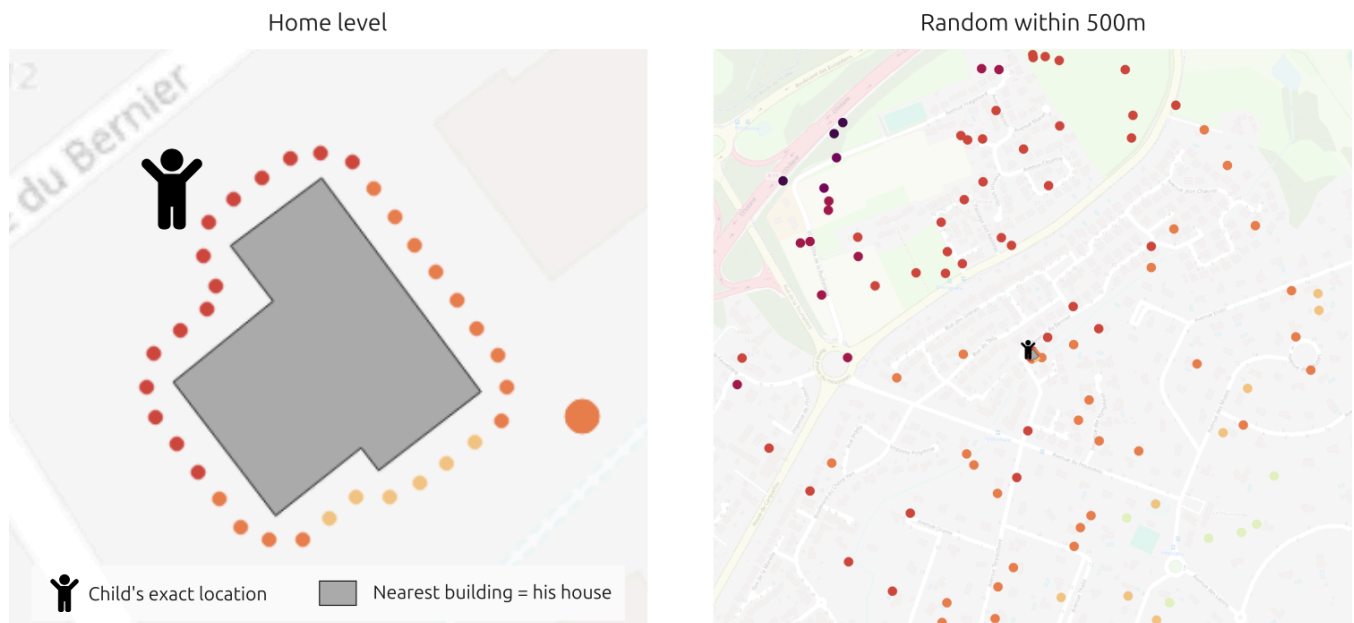
Objectifs

L'objectif de ce travail est de pouvoir estimer l'exposition au bruit des enfants de la cohorte ELFE. Pour ce faire, deux niveaux d'analyse ont été définis :

1. Niveaux sonores calculés autour du bâtiment ou l'enfant vit. Pour cela, on réalise les calculs sur le bâtiment le plus proche de l'enfant, qu'on suppose être son domicile.

2. Niveaux sonores calculés dans un rayon de 500m autour de l'enfant. On se base pour cela sur 100 points récepteurs, tirés aléatoirement dans cette zone tampon (remarque : ces points ne peuvent se superposer à un bâtiment ou à l'emprise d'une route).

Le schéma ci-dessous illustre ces deux niveaux d'analyse



En terme de sources sonores, on évalue :

1. Le bruit émis par le trafic routier, calculé pour l'occasion à l'aide du logiciel NoiseModelling (développé par l'UMRAE, en charge de ce travail),
2. Pour peu que les récepteurs créés intersectent ces sources :
 1. Les niveaux sonores du trafic aérien,
 2. Les niveaux sonores du trafic ferroviaire.

Données d'entrée

Pour réaliser ce travail, on se base sur les données d'entrée suivantes :

- La liste des enfants, identifiés via un code unique **FID** et localisables via un couple de coordonnées **X** et **Y**. Cette donnée est fournie par les partenaires de l'INSERM qui pilotent ce projet,
- La base de données ouverte [OpenStreetMap](https://www.openstreetmap.org/) (OSM), qui servira notamment à obtenir les bâtiments, les types d'occupation du sol ainsi que le modèle numérique d'élévation,
- Les Plans d'Exposition au Bruit (PEB) aérien, établis par la [DGAC](https://www.dgac.fr/) et accessibles ici <https://geoservices.ign.fr/services-web-experts-transports#2314>
- Les Cartes de Bruit Stratégique (CBS) des grandes lignes ferroviaires, calculées dans le cadre du projet Plamade pour le compte de l'état français (voir <https://hal.science/hal-03848495/document>)

Outils

La chaîne de traitement ici mise en œuvre repose principalement sur les deux outils libres et gratuits [GeoClimate](#) et [NoiseModelling](#).

- GeoClimate est utilisé pour télécharger et mettre en forme les données issues d'OpenStreetMap, qui alimentent ensuite NoiseModelling.
- NoiseModelling est utilisé pour calculer les niveaux sonores dus au trafic routier, au niveau des récepteurs (maison et aléatoire).
- L'ensemble est mis en musique via un script bash

Remarque : Les traitements et scripts présentés ci-dessous sont réalisés dans un environnement Linux ([Ubuntu](#) 24.10).

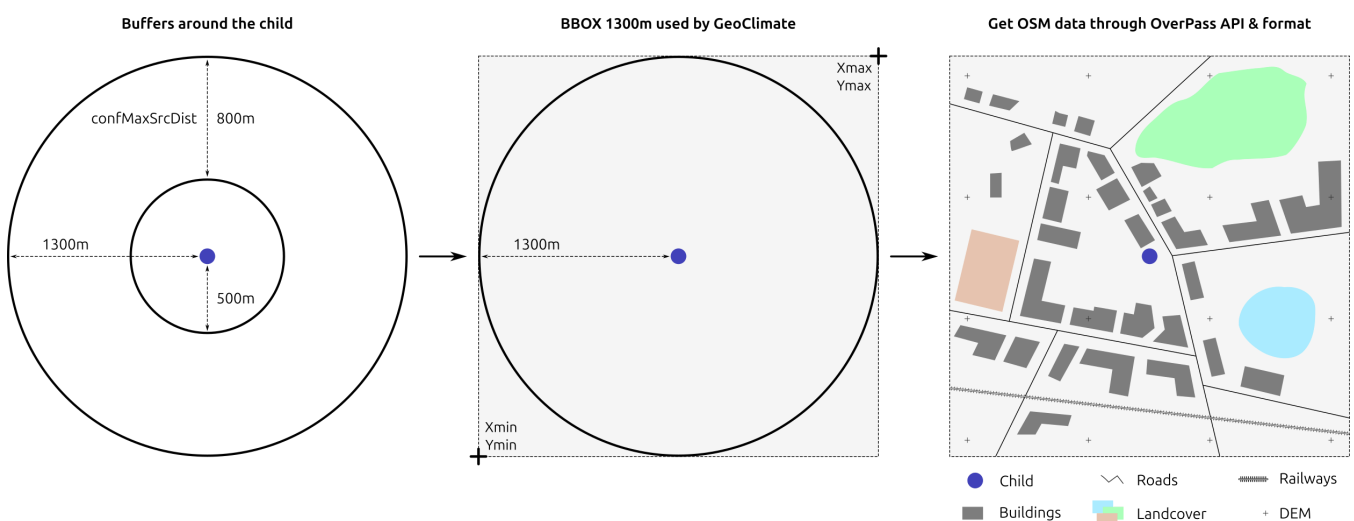
1. Préparation des données

Définition des zones de travail

La demande initiale est de pouvoir calculer les niveaux sonores autour du logement de l'enfant ainsi que dans un périmètre de 500m alentour.

Pour que les niveaux sonores calculés soient cohérents avec la réalité, nous estimons qu'une source sonore peut avoir une influence sur un récepteur jusqu'à 800m de distance (paramètre `confMaxSrcDist` dans NoiseModelling - [voir](#)). Dès lors, en bordure des 500m, il est nécessaire d'aller chercher les données jusqu'à 1300m autour de l'enfant afin de pouvoir couvrir cette zone de contribution acoustique.

La récupération des données issues d'OpenStreetMap se fait via GeoClimate. En entrée, cette librairie attend une enveloppe (aussi appelée *Bounding Box* - BBOX). Celle-ci est basée sur le buffer de 1300m et définie via le couple de coordonnées "Sud-Ouest" et "Nord-Est" (`Xmin`, `Ymin` et `Xmax`, `Ymax`). Sur la base de ces deux points, GeoClimate envoie une requête à l'[API OverPass](#), qui va elle même récupérer les données d'OSM. GeoClimate se charge ensuite de formater les données ainsi obtenues.



Traitements dans une base de données

Cette phase consiste à importer les données d'entrées (enfants, réseaux routiers, CBS des avions et des trains) et à réaliser des traitements pour isoler les données utiles, autour des enfants, dans une limite de 1300m.

Pour ce faire, on va utiliser une base de données [H2GIS](#) et exécuter le fichier `script.sql` présent dans le dossier `2024_CHILD/bdd/`.

Processus

1. Import des données d'entrée :

1. enfants (avec seulement deux informations : les coordonnées du point (Lat/Long - WGS84) + l'identifiant unique de l'enfant `FID`) → reprojection en Lambert 93 ([EPSG:2154](#))
2. cartes de bruit de train (couche `cbs_f_2154` issue du projet Plamade)
3. cartes de bruit des avion (Plans d'Exposition au Bruit)

2. Génération de buffers autour des enfants :

1. couche `BUFFER_500_2154` pour des buffers de 500m
2. couche `BUFFER_1300_2154` pour des buffers de 1300m

3. Pour la route

1. Suppression des tunnels
2. Intersection des tronçons avec les buffers de 1300m autour des enfants. Les résultats sont stockés dans la table `trafic_predit_1300` puis exportés dans le fichier `.../input_data/Predicted_traffic/TRAFFIC_PREDICT_1300.geojson`.

4. Pour l'avion

1. Suppression des géométries ayant des valeurs non nécessaires (ex: `INDLDENINT = 'Emprise'`)
2. Ajout de 2 champs pour coder les niveaux sonores :
 1. `AERO_TXT` au format texte, qui concatène les bornes basse `INDLDENINT` et haute `INDLDENEXT` (ex: `INDLDENINT = 55` et `INDLDENEXT = 59` → `AERO_TXT = 55-59`)
 2. `AERO` qui stocke la moyenne des bornes haute et basse, exprimée en dB (ex: $(55+59)/2$ → `AERO = 57`)
3. Intersection de la couche résultante (`AVION_2154`) avec les buffers de 500m autour des enfants. Les résultats sont stockés dans la table `avion_buffer` puis exportés dans le fichier `.../input_data/avion/avion.geojson`.

5. Pour le train

1. Séparation des CBS (de `TYPE = A`) en deux couches distinctes pour le jour (`INDICETYPE = 'LD'`) et la nuit (`INDICETYPE = 'LN'`)
2. Formatage des catégories (champ `category`) en niveaux sonores exprimés en texte (champ `LDEN`) et en décibel (dB) (champ `LDEN_DB`) (ex: `category = Lden5559` va donner `LDEN_DB = 57` et `LDEN = 55-59`). Idem pour LNIGHT. Voir le détail dans les tableaux ci-dessous.

3. Intersection de deux couches résultantes (`train_lden` et `train_lnight`) avec les buffers de 500m autour des enfants. Cela permet donc de ne garder que les zones à proximité des enfants. Les résultats sont stockés dans deux tables `train_lden_buffer` et `train_lnight_buffer`.

Tableau de correspondance pour le formatage des isosurfaces de train en LDEN

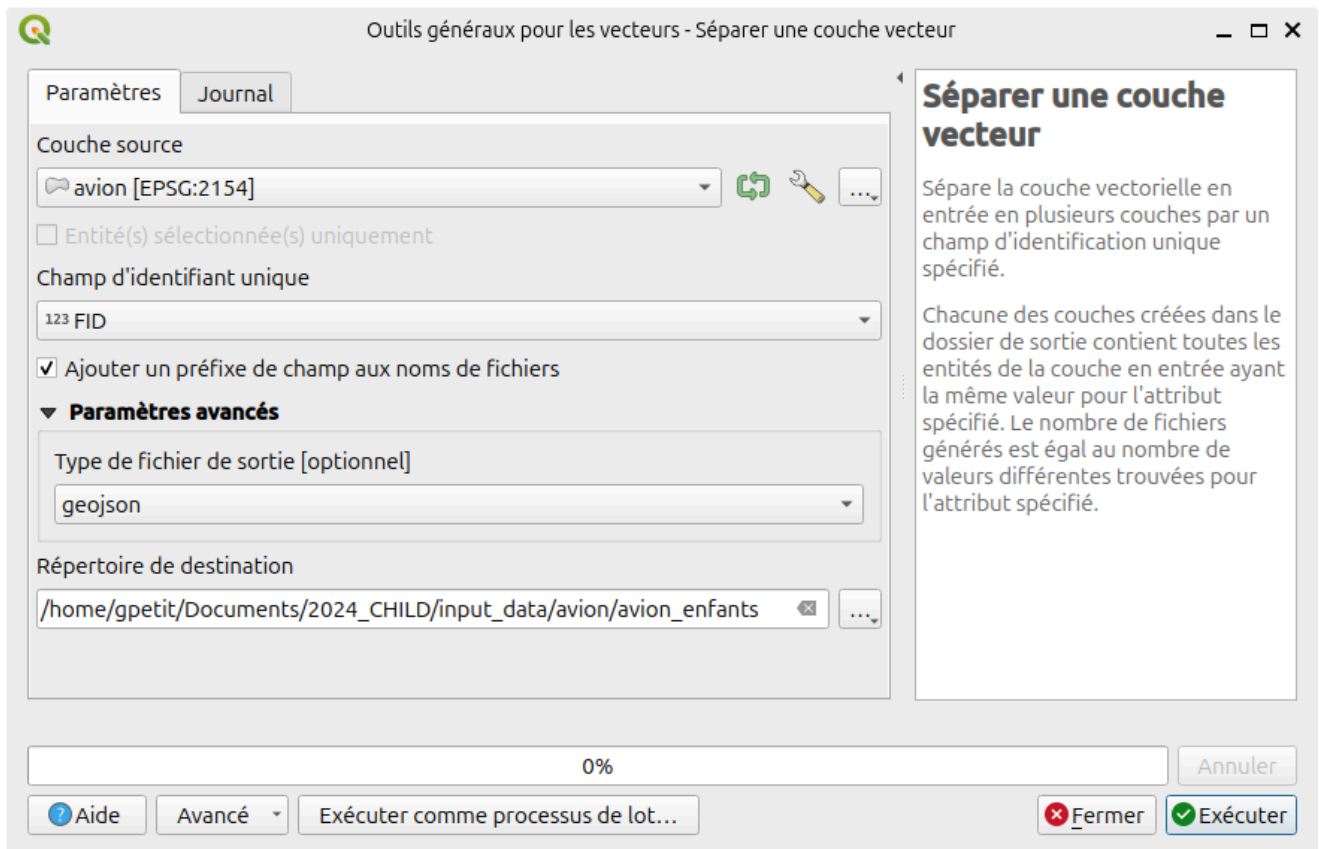
category	LDEN	LDEN_DB
Lden5559	55-59	57
Lden6064	60-64	62
Lden6569	65-69	67
Lden7074	70-74	72
LdenGreaterThan75	sup 75	75

Tableau de correspondance pour le formatage des isosurfaces de train en LNIGHT

category	LNIGHT	LNIGHT_DB
Lnight5054	50-54	52
Lnight5559	55-59	57
Lnight6064	60-64	62
Lnight6569	65-69	67
LnightGreaterThan70	sup 70	70

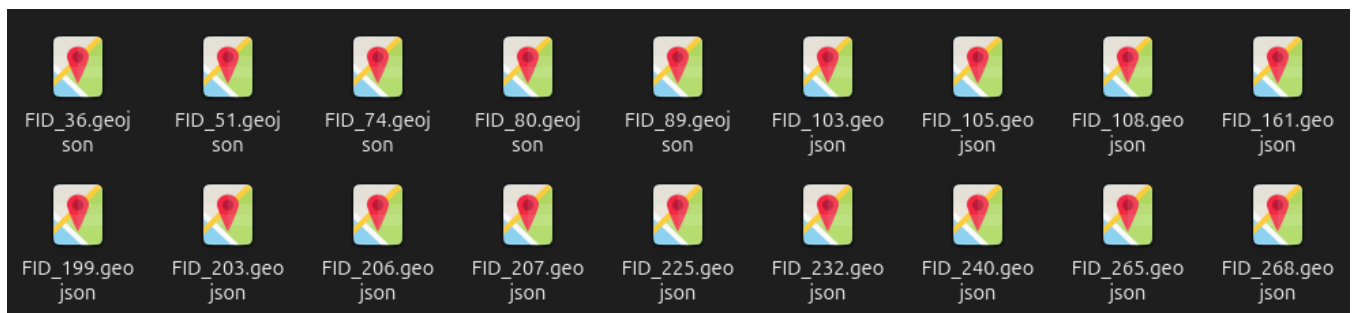
Séparation des fichiers par enfants

À l'issue des traitements SQL dans la base de données, on obtient des couches de ROUTE (`trafic_predit_1300`), AVION (`avion_buffer`) ou TRAIN (`train_lden_buffer` et `train_lnight_buffer`), à l'échelle nationale (tous les enfants sont ensembles). L'objectif de cette étape est de générer des fichiers de ROUTE, AVION ou TRAIN par enfants. Pour cela, on va utiliser QGIS qui dispose d'une interface `Séparer une couche vecteur` (via le menu `Vecteur/Outils de gestion des données`).



Dans l'exemple ci-dessus, on va séparer la couche `avion` sur la base du champ `FID` (l'identifiant de l'enfant). Les fichiers seront exportés dans le dossier `.../2024_CHILD/input_data/avion/avion_enfants/` au format `.geojson`.

On obtient alors les fichiers `FID_xxx.geojson` ci-dessous



On réalise cette opération pour les 3 données suivantes :

- ROUTE : la couche `trafic_predit_1300` sera décomposée et stockée dans le dossier `input_data/trafic_enfants`
- AVION : la couche `avion_buffer` sera décomposée et stockée dans le dossier `input_data/avion/avion_enfants`
- TRAIN
 - la couche `train_lden_buffer` sera décomposée et stockée dans le dossier `input_data/train/train_enfants_lden`
 - la couche `train_lnight_buffer` sera décomposée et stockée dans le dossier `input_data/train/train_enfants_lnight`

Conformité des fichiers .geojson

Pour une raison inconnue, il semble que l'export `.geojson` de QGIS insère une ligne qui pose soucis pour la suite des traitements dans NoiseModelling. La ligne en question est la suivante :

```
"xy_coordinate_resolution": 1e-15,
```

```
{
  "type": "FeatureCollection",
  "name": "FID_36",
  "crs": { "type": "name", "properties": { "name": "urn:ogc:def:crs:EPSG::2154" } },
  "xy_coordinate_resolution": 1e-15,
  "features": [
    { "type": "Feature", "properties": { "FID": 36, "AERO_TXT": "55-50", "AERO": 52.5 },
    "geometry": { "type": "MultiPolygon", "coordinates": [ [ [ [ 944311.667655157274567,
6890016.351212566718459 ], [ 944276.084615593310446, 6889972.993153406307101 ], [
944251.515380977187306, 6889952.829703649505973 ], [ 944311.667655157274567,
6890016.351212566718459 ] ] ] ] } }
  ]
}
```

Nous allons donc devoir supprimer cette ligne, dans l'ensemble des fichiers `.geojson` généré via cette méthode de séparation de couche. Pour cela, il suffit d'exécuter la commande ci-dessous dans un terminal :

```
find /home/gpetit/Documents/2024_CHILD/input_data/avion/avion_enfants/ -name "*.geojson" -
exec sed -i 's/"xy_coordinate_resolution": 1e-15,//g' {} \;
```

Remarque : Le dossier `.../avion/avion_enfants/` est à adapter en fonction des données à corriger.

Une fois fait, les fichiers `FID_xxx.geojson` se présentent sous la forme ci-dessous et sont acceptés par NoiseModelling.

```
{
  "type": "FeatureCollection",
  "name": "FID_36",
  "crs": { "type": "name", "properties": { "name": "urn:ogc:def:crs:EPSG::2154" } },

  "features": [
    { "type": "Feature", "properties": { "FID": 36, "AERO_TXT": "55-50", "AERO": 52.5 },
    "geometry": { "type": "MultiPolygon", "coordinates": [ [ [ [ 944311.667655157274567,
6890016.351212566718459 ], [ 944276.084615593310446, 6889972.993153406307101 ], [
944251.515380977187306, 6889952.829703649505973 ], [ 944311.667655157274567,
6890016.351212566718459 ] ] ] ] } }
  ]
}
```

Renommage des fichiers

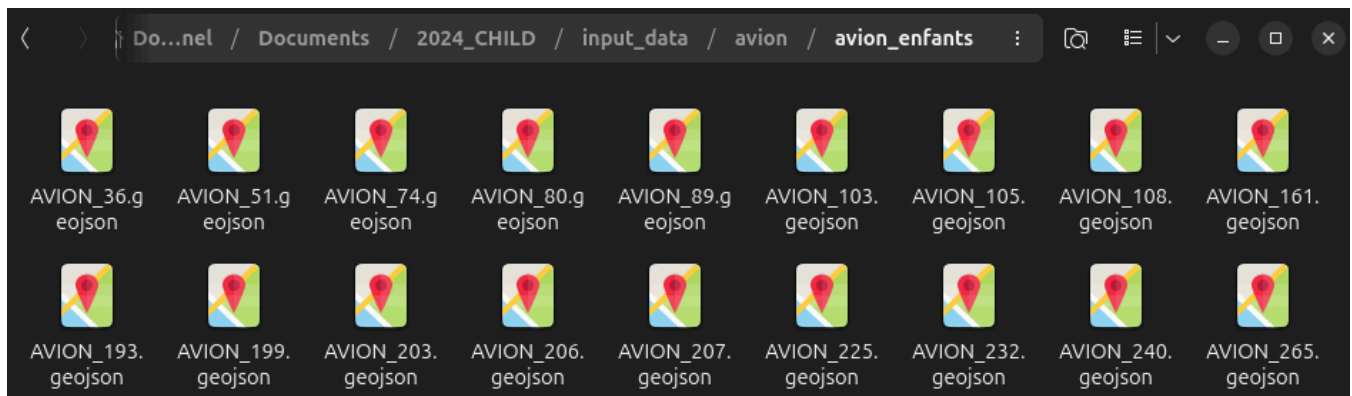
Une fois la séparation des enfants réalisée, il faut renommer les fichiers `FID_xxx.geojson` en fonction de la nature des données :

- Pour les routes : `FID_xxx.geojson` → `ROUTE_xxx.geojson`
- Pour les avions : `FID_xxx.geojson` → `AVION_xxx.geojson`
- Pour les trains :
 - DEN : `FID_xxx.geojson` → `TRAIN_LDEN_xxx.geojson`
 - Night : `FID_xxx.geojson` → `TRAIN_LNIGHT_xxx.geojson`

Pour cela, il suffit d'exécuter la commande suivante dans un terminal

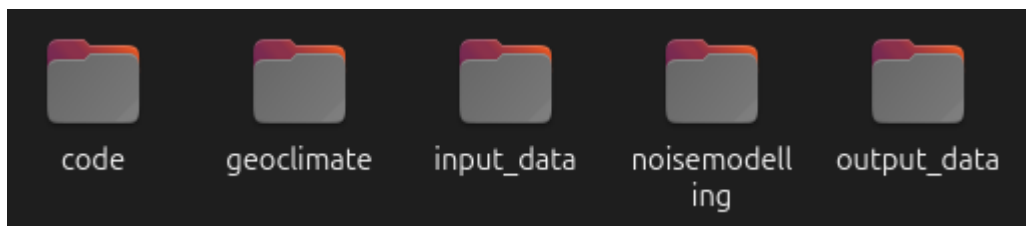
```
# On va dans le dossier contenant les fichiers .geojson à renommer
# Ici on prend l'exemple des avions
cd ../../input_data/avion/avion_enfants
# On remplace FID par AVION
for file in FID_*.geojson; do mv "$file" "${file/FID/AVION}"; done
```

On obtient alors les fichiers suivants



2. Organisation des données et fichiers

Dans le dossier de travail, on retrouve les 5 dossiers suivants :



Dossier code

Dossier contenant l'ensemble des fichiers permettant de faire fonctionner NoiseModelling. On y retrouve notamment :

- des fichiers `.groovy` propres à ce projet CHILD,

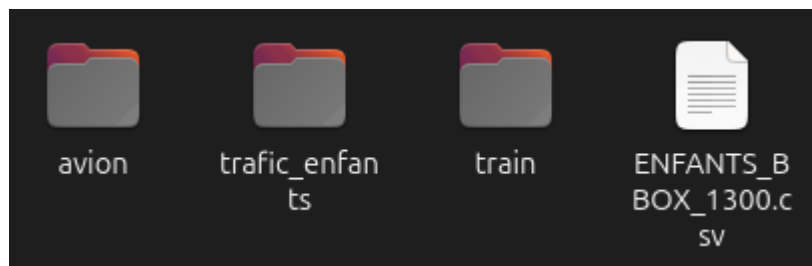
- le fichier `run_child.sh` qui permettra d'exécuter l'ensemble de la chaîne de traitement.

Dossier `geoclimate`

Dossier contenant la librairie [GeoClimate](#) (fichier `GeoClimate_Tools-1.0.3-SNAPSHOT.jar`) nécessaire pour appeler et formater les données issues d'[OpenStreetMap](#).

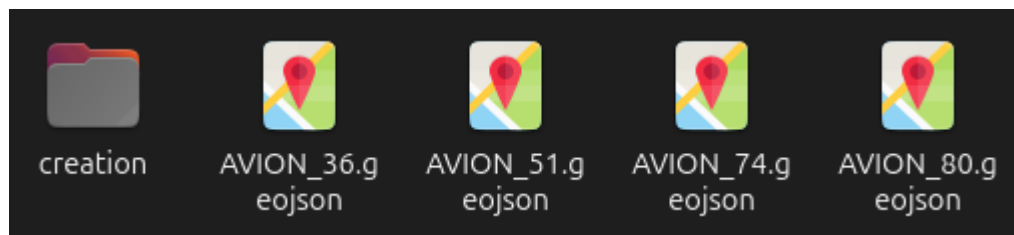
Dossier `input_data`

Dossier contenant l'ensemble des données d'entrée

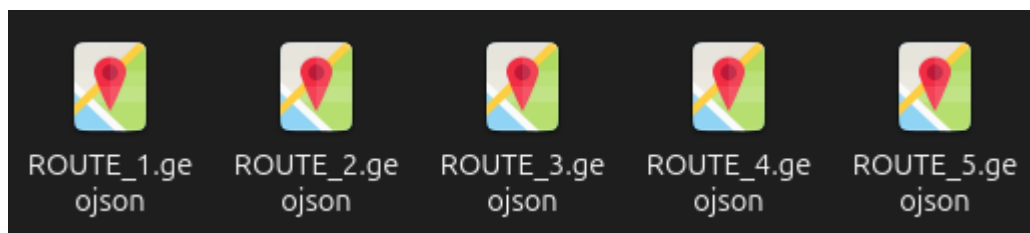


On y retrouve :

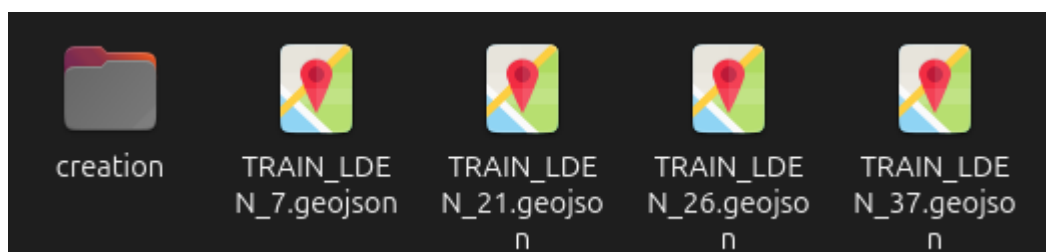
- Dossier `avion` et le sous-dossier `avion_enfants` dans lequel on retrouve l'ensemble des fichiers `AVION_XXX.geojson` ainsi que le dossier vide `creation` (voir explication plus bas) qui sera alimenté au fur et à mesure des calculs.



- Dossier `trafic_enfants` dans lequel on retrouve les fichiers `ROUTE_XXX.geojson`



- Dossier `train` dans lequel on retrouve les deux sous-dossiers `train_enfants_lden` (voir illustration ci-dessous) et `train_enfants_lnight`, respectivement pour les niveaux Lden et Lnight, avec dans les deux cas, les fichiers respectifs `TRAIN_LDEN_XXX.geojson` ou `TRAIN_LNIGHT_XXX.geojson` ainsi qu'un dossier vide `creation` qui sera alimenté au fur et à mesure des calculs.



- Fichier `ENFANTS_BBBOX_1300.csv` contenant la liste des enfants à traiter (identifiant unique `FID`), avec la géométrie du point `GEOM_POINT` (en WGS84) et l'enveloppe de 1300m `BBBOX` (en WGS84).

```

1 FID;GEOM_POINT;BBOX
2 1;POINT (-0.6339519999992644 44.845792999999865);44.83356152760585,-0.6496265847826307,44.85802230645642,-0.6182705883621846
3 2;POINT (2.72951800000029107 50.6487029999996964);50.636995369088034,2.7112359875834255,50.66040773552516,2.7478085401857504
4 3;POINT (-0.69548100000033018 44.792964999999935);44.780725059517216,-0.7111274731315982,44.80520278284394,-0.6798277138320571
5 4;POINT (2.44682300000005953 48.9047479999997994);48.892974369829574,2.4292177546197333,48.9165189144637,2.4644362101844264
6 5;POINT (-0.75226800000053469 46.923153999999543);46.91090536265822,-0.7685175168753041,46.93540030048447,-0.7360111240450373

```

Ajout du dossier vide **creation**

Dans les dossiers `avion_enfants`, `train_enfants_lden` et `train_enfants_lnight` on retrouve respectivement les fichiers `AVION_xxx.geojson`, `TRAIN_LDEN_XXX.geojson` ou `TRAIN_LNIGHT_XXX.geojson` lorsque qu'il y a intersection avec le buffer de 500m autour de l'enfant.

S'il n'y a pas d'intersection, alors il n'y a pas de fichier.

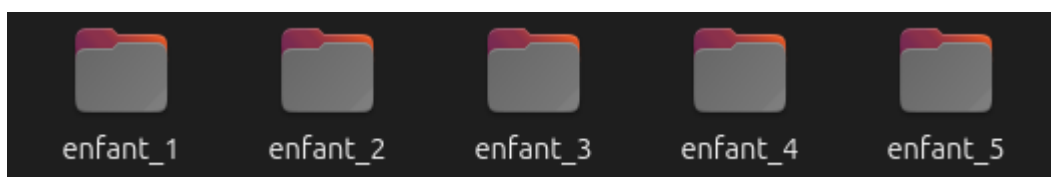
Dans le script général `run_child.sh` (voir plus bas), lorsqu'on calcule les niveaux sonores au niveau des récepteurs, NoiseModelling attend les fichiers relatifs aux avions et trains. Si les fichiers n'existent pas, alors NoiseModelling va renvoyer une erreur et s'arrêter. Pour éviter cela, un mécanisme de substitution a été mis en place : si le fichier n'existe pas dans le dossier, alors le script génère le fichier attendu, dans le sous-dossier `creation` et y insère une valeur nulle. De ce fait, le fichier `.geojson` est conforme (bien que vide) et NoiseModelling pourra l'importer sans faire planter le script (l'absence de géométrie indiquant simplement que l'enfant ne vit pas à moins de 500m du train ou de l'avion).

Dossier **noisemodelling**

Dossier contenant NoiseModelling 5.0 dans les sous-dossiers `noisemodelling/5.0/NoiseModelling_without_gui/`.

Dossier **output_data**

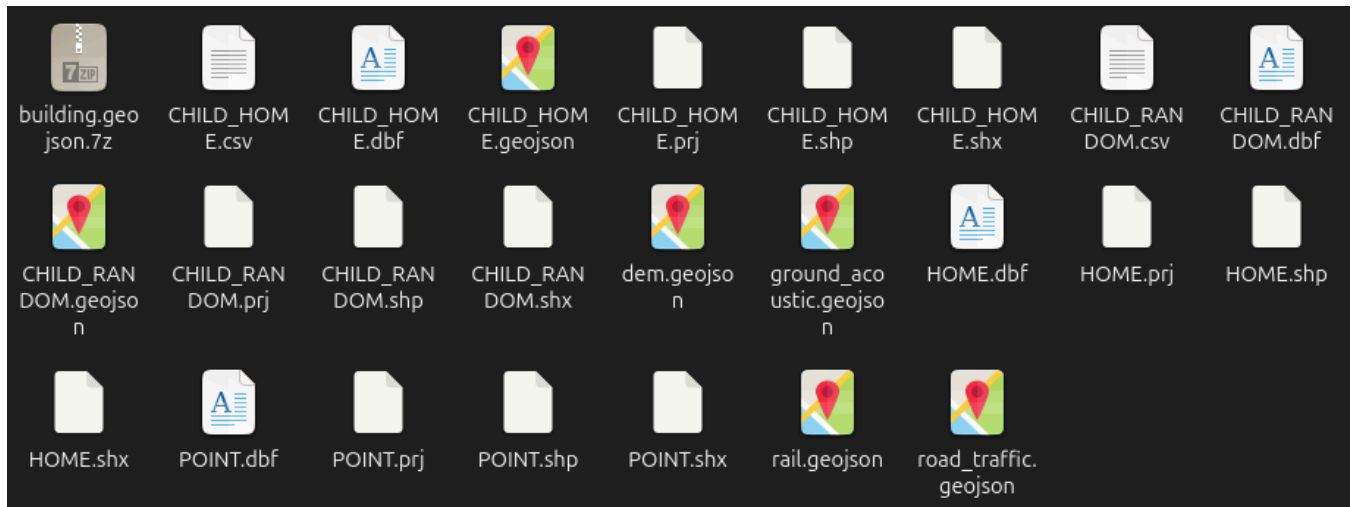
Dossier vide, qui sera amené à recevoir au fur à mesure de l'avancé des calculs, l'ensemble des dossiers résultants `enfant_xxx`, propres à chaque enfants.



Pour chacun des enfants on retrouve les fichiers suivants :

- `building.geojson.7z` : le fichier compressé contenant le fichier `building.geojson` de l'ensemble des bâtiments dans la zone de 1300m autour de l'enfant, issus d'OSM (GeoClimate) (voir explication plus bas, dans la section `Compression des bâtiments`),
- `CHILD_HOME.shp` : niveaux sonores sur les récepteurs autour de la maison ou habite l'enfant,
- `CHILD_RANDOM.shp` : niveaux sonores sur les 100 récepteurs, répartis aléatoirement dans une zone de 500m autour de la maison ou habite l'enfant,
- `dem.geojson` : données altimétriques issues d'OSM (GeoClimate),
- `HOME.shp` : bâtiment, issu de la couche `building.geojson`, le plus proche de l'enfant. Donc supposément sa maison,

- `POINT.shp` : le point de l'enfant,
- `rail.geojson` : réseau ferré, issu d'OSM (GeoClimate),
- `road_traffic.geojson` : réseau routier, issu d'OSM (GeoClimate). À noter que cette couche n'est pas utilisée par NoiseModelling étant donné que nous la substituons par la couche `ROUTE_xxx.geojson` pré-traitée en amont et stockée dans le dossier `input_data/trafic_enfants/`.



Modèle de données

Dans les deux tables `CHILD_HOME` et `CHILD_RANDOM` on retrouve les colonnes suivantes :

Colonne	Type	Définition
<code>THE_GEOM</code>	Geometry	Point situant le récepteur (coordonnées X et Y, exprimées en Lambert 93 (EPSG:2154))
<code>FID</code>	Integer	Identifiant unique de l'enfant
<code>IDRECEIVER</code>	Integer	Identifiant unique du récepteur
<code>ROAD_DEN</code>	Double	Niveau sonore (en dB(A)) émis par le trafic routier sur toute la journée (Day-Evening-Night)
<code>ROAD_DAY</code>	Double	... le jour (06-18h)
<code>ROAD_EVE</code>	Double	... le soir (18-22h)
<code>ROAD_NIGHT</code>	Double	... la nuit (22-06h)
<code>RAIL_DEN</code>	Double	Niveau sonore (en dB) émis par les lignes ferroviaires à grande vitesse, sur toute la journée *
<code>RAIL_NIGHT</code>	Double	... la nuit *
<code>AERO</code>	Text	Classe de bruit issu du trafic aérien (ex: 55-59) *

Colonne	Type	Définition
AERO_DB	Integer	Niveau sonore (en dB) déduit de AERO (ex: 55-59 donne 57) *

* Si le récepteur n'intersecte pas la zone en question, alors la cellule est laissée vide

3. Exécution des calculs

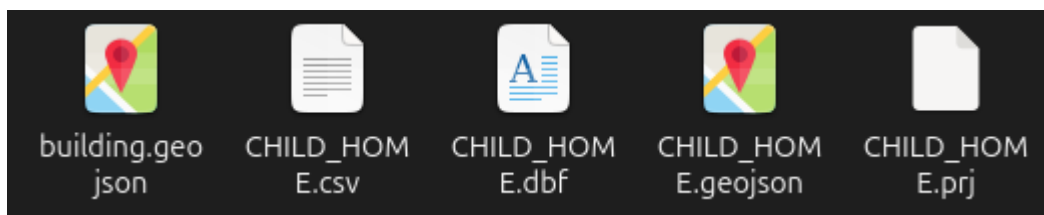
Dans un terminal, aller dans le dossier où se trouve le fichier `run.child.sh`, puis exécuter la commande

```
cd /home/gpetit/Documents/2024_CHILD/child_production/code/
bash run_child.sh
```

4. Mise en forme des résultats

Compression des bâtiments

Dans chacun des dossiers résultants, se trouve un fichier `building.geojson`, qui potentiellement prend beaucoup de place (environ 90% du poids global du dossier).

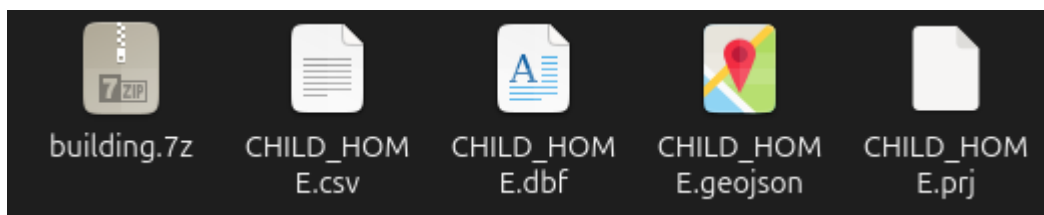


Étant donné que cette donnée n'a pas vocation à être réutilisée dans l'immédiat, on va pouvoir la compresser dans un fichier `.7z` (qui a un meilleur taux de compression que le `.zip`).

Pour cela, il suffit d'exécuter le script `compression_building_geojson.sh`, qui va chercher de manière récursive tous les fichiers présents dans le dossier `output_data`, avec la commande suivante (dans un terminal) :

```
cd /home/gpetit/Documents/2024_CHILD/
bash compression_building_geojson.sh
```

Une fois le traitement terminé, l'ensemble des fichiers `building.geojson` sera remplacé par les fichiers `building.geojson.7z`



Unification des enfants

Une fois les enfants calculés, il est nécessaire de les rassembler dans des fichiers communs. Pour cela, on va faire appel à la librairie [ogr2ogr](#).

Dans un terminal, aller dans le dossier `output_data` puis exécuter la commande ci-dessous

```
cd output_data
find $(pwd) -type f -name 'CHILD_HOME.shp' -exec ogr2ogr -f "SQLite"
CHILD_HOME_MERGED.sqlite {} -append \;
```

Cette commande permet de chercher l'ensemble des fichiers `CHILD_HOME.shp` et de les fusionner dans un nouveau fichier `CHILD_HOME_MERGED.sqlite` qui sera stocké à la racine du dossier.

On répète ensuite la commande avec le fichier `CHILD_RANDOM.shp`

```
find $(pwd) -type f -name 'CHILD_RANDOM.shp' -exec ogr2ogr -f "SQLite"
CHILD_RANDOM_MERGED.sqlite {} -append \;
```

On préfère ici d'exporter dans un fichier `.sqlite` car les formats `.shp`, `.geojson` ou `.gpkg` sont trop longs à générer et pèsent finalement très lourd.

Enfin, on reproduit la démarche pour les fichiers `CHILD_HOME.csv` et `CHILD_RANDOM.csv` avec les commandes suivantes :

```
find . -type f -name 'CHILD_HOME.csv' -exec head -n 1 {} \; | head -n 1 >
CHILD_HOME_MERGED.csv && find . -type f -name 'CHILD_HOME.csv' -exec tail -n +2 {} \; >>
CHILD_HOME_MERGED.csv

find . -type f -name 'CHILD_RANDOM.csv' -exec head -n 1 {} \; | head -n 1 >
CHILD_RANDOM_MERGED.csv && find . -type f -name 'CHILD_RANDOM.csv' -exec tail -n +2 {} \; >>
CHILD_RANDOM_MERGED.csv
```